Ex no:1
Date : 30/7/26

STUDY OF VARIOUS COMMANDS USED IN LINUX AND WINDOWS

AIM:
To study about various networking commands on Linux operating system.

BASIC LINUX COMMANDS

1. pwd

Description: Points the current working directory.

Output:

```
$ pwd

/home/liveuser
```

2. mkdir Bedora

Description: Create a new directory.

Output:

```
$ mkdir Bedora

$ ls

Desktop  Documents  Downloads  Bedora
```

3. cd Bedora

Description: Changes the current directory.

Output:

```
$ cd Bedora

$ pwd

/home/liveuser/Bedora
```

4. touch example.txt

Description: Create an empty file.

Output:

```
$ touch example.txt

$ ls

example.txt
```

5. echo "Hi" > example.txt

Description: Write text into a file.

Output:

```
$ echo "Hi" > example.txt
```

6. cat example.txt

Description: Display the contents of a file.

Output:

```
$ cat example.txt

Hi
```

7. grep "Hi" example.txt

Description: Searches for a specific pattern in a file.

Output:

```
$ grep "Hi" example.txt

Hi
```

8. wc example.txt

Description: Display the number of lines, words and bytes in a file.

Output:

```
$ wc example.txt

1  1  3  example.txt
```

9. wc -l example.txt

Description: Counts the number of lines in a file.

Output:

```
$ wc -l example.txt

1  example.txt
```

10. wc -w example.txt

Description: Counts the number of words in a file.

Output:

```
$ wc -w example.txt

1  example.txt
```

11. wc -c example.txt

Description: Counts the number of bytes in a file.

Output:

```
$ wc -c example.txt

3  example.txt
```

12. wc -m example.txt

Description: Counts the number of characters in a file.

Output:

```
$ wc -m example.txt

3  example.txt
```

13. wc -L example.txt

Description: Displays the length of the longest line in a file.

Output:

```
$ wc -L example.txt

2  example.txt
```

BASIC NETWORKING COMMANDS

1. hostname

Description: Displays the system's hostname.

Output:

```
$ hostname

localhost-live
```

2. route

Description: Displays the routing table.

Output:

```
$ route

Kernel IP routing table
```

3. netstat

Description: Shows network connections and routing information.

Output:

```
$ netstat

Active Internet connections
```

4. ip address show

Description: Displays IP address and network interface details.

Output:

```
$ ip address show

1: lo:

2: enp0s3:
```

5. nslookup

Description: Finds the IP address of a domain.

Output:

```
$ nslookup www.google.com

Name: www.google.com

Address: 142.250.xxx.xxx
```

6. ping

Description: Tests connectivity to a host.

Output:

```
$ ping www.google.com

64 bytes from ...
```

7. arp

Description: Displays the ARP cache.

Output:

```
$ arp -a

gateway (10.0.2.2)
```

8. traceroute

Description: Displays the route taken by packets to reach a destination.

Output:

```
$ traceroute www.google.com

1  gateway
```

2 . . . .

RESULT:

The basic Linux and networking commands were executed successfully and expected outputs were obtained.

Exno:2

EXPLORING THE IP

AIM:

Study of Different Types of Network Cables

OBJECTIVE

To study and understand the different types of network cables used in computer networking, their categories, data transmission speeds, advantages, disadvantages, and applications.

INTRODUCTION

Network cables are physical communication media used to connect computers, network devices, servers, and other networking equipment. They are used to transmit data from one device to another. Different types of cables are selected depending on factors such as transmission speed, distance, cost, bandwidth, and environmental conditions.

The major types of network cables are:

1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) Cable
3. Coaxial Cable
4. Fibre Optic Cable

---

1. UNHSHIELDED TWISTED PAIR (UTP) CABLE

UTP cable consists of pairs of insulated copper wires twisted together to reduce electromagnetic interference. It is one of the most commonly used cables in LAN networks.

Categories and Applications

Category	Maximum Data Transmission	Application / Use
Category 3 (Cat 3)	Up to 10 Mbps	10Base-T Ethernet and telephone networks
Category 5 (Cat 5)	Up to 100 Mbps	Fast Ethernet
Category 5e (Cat 5e)	Up to 1 Gbps	Gigabit Ethernet
Category 6 (Cat 6)	Up to 10 Gbps	Gigabit Ethernet and high-speed networks

Category 6a (Cat 6a)	Up to 10 Gbps	10G Ethernet

Advantages

- Low cost and economical.
- Easy to install and maintain.
- Lightweight and flexible.
- Smaller overall diameter.
- Widely available.

Disadvantages

- More susceptible to electromagnetic interference (EMI).
- Signal quality may decrease over long distances.
- Provides less protection against external noise compared to shielded cables.

Applications

- Local Area Networks (LAN).
- Ethernet connections.
- Computer-to-switch connections.
- Telephone and data communication networks.

---

2. SHIELDED TWISTED PAIR (STP) CABLE

STP cable is a twisted-pair cable that has an additional metallic shield around the wires. The shielding provides better protection against electromagnetic interference and electrical noise.

Categories and Applications

Category	Maximum Data Transmission	Application / Use
Cat 6	Up to 10 Gbps	Gigabit Ethernet
Cat 6a	Up to 10 Gbps	10G Ethernet
Cat 7	Up to 10 Gbps	High-speed Ethernet and data centres

Advantages

- Better protection from electromagnetic interference.
- Less affected by electrical noise.
- Provides reliable data transmission.
- Suitable for environments with high electrical interference.

- Supports high-speed networking.

Disadvantages

- More expensive than UTP.
- Installation is comparatively difficult.
- Thicker and less flexible.
- Requires proper grounding for effective shielding.

Applications

- Data centres.
- Industrial networks.
- High-speed Ethernet networks.
- Areas with high electromagnetic interference.

---

3. COAXIAL CABLE

Coaxial cable consists of a central copper conductor, an insulating layer, a metallic shield, and an outer protective jacket. It provides good protection against interference and has been widely used for television and communication systems.

Types of Coaxial Cable

Type	Typical Application
RG-6	Cable television and high-speed internet
RG-59	CCTV and television systems
RG-11	Long-distance cable television and broadband connections

Maximum Data Transmission

Coaxial cable can support data rates ranging from approximately **10 Mbps to 100 Mbps**, depending on the cable type and network technology.

Advantages

- Good resistance to electromagnetic interference.
- Reliable signal transmission.
- Good bandwidth.
- Low signal loss.
- Durable and versatile.

Disadvantages

- Bulky compared with twisted-pair cables.
- More difficult to install.
- Higher cost in some applications.
- Limited flexibility.

Applications

- Cable television networks.
- Broadband internet connections.
- CCTV systems.
- Radio and communication systems.

---

4. FIBRE OPTIC CABLE

Fibre optic cable transmits data in the form of light signals through thin strands of glass or plastic fibre. It provides very high bandwidth and is suitable for long-distance and high-speed communication.

Types of Fibre Optic Cable

Type	Description	Application
Single-Mode Fibre	Uses a single light path and supports long-distance communication	Long-distance networks, telecom and WAN
Multi-Mode Fibre	Uses multiple light paths and is suitable for shorter distances	LANs, data centres and campus networks

Data Transmission

Fibre optic cables can support **very high data rates, including 100 Gbps and beyond**, depending on the equipment and fibre technology used.

Advantages

- Very high transmission speed.
- High bandwidth.
- Supports long-distance communication.
- Immune to electromagnetic interference.
- Provides better security than copper cables.
- Low signal attenuation.
- Lightweight compared with many copper cable systems.

Disadvantages

- Higher installation cost.
- Requires skilled technicians for installation and maintenance.
- Fibre can be more delicate than copper cables.
- Special equipment is required for splicing and testing.

Applications

- Internet backbone networks.
- Data centres.
- Telecommunication networks.
- Long-distance communication.
- High-speed LAN and WAN networks.

COMPARISON OF DIFFERENT NETWORK CABLES

Cable Type	Medium	Speed	Interference Protection	Cost	Common Applications
UTP	Copper	Up to 10 Gbps	Low to Medium	Low	LAN, Ethernet
STP	Shielded Copper	Up to 10 Gbps	High	Medium	Data centres, industrial networks
Coaxial	Copper	Up to 100 Mbps*	High	Medium	TV, CCTV, broadband
Fibre Optic	Glass/Plastic Fibre	100 Gbps and beyond*	Very High	High	Internet, telecom, data centres

*Actual speed depends on the specific cable, equipment, standard, and network configuration.

RESULT

Thus, the different types ofnetwork cable were studied and verified.

EXP NO.: 3
DATE:

CISCO PACKET TRACER

AIM:

To analyse the behaviour of network devices using **CISCO PACKET TRACER SIMULATOR**.

PROCEDURE:

1. From the network component box, click and drag the following components: **4 generic PCs and one switch**.
2. To make new connections:
    - **a.** Click on the **Copper Straight-Through cable**.
    - **b.** Select a port on the PC and connect it to the switch using the cable. Then the **Link LED** should glow in green.
    - **c.** Similarly, link the other three computers.
3. Click on the connected PC, go to the **Desktop** tab, click on **IP Configuration**, and enter an **IP address and subnet mask**.
4. Click on the **PDU (message icon)** and send it to the PCs.
5. Observe the flow of the PDU from source PC to destination PC by selecting the **Real-Time/Simulation mode**.

HOW TO DOWNLOAD AND INSTALL CISCO PACKET TRACER:

Cisco provides Packet Tracer through its learning platforms. Cisco's installation instructions state that Packet Tracer is available for **Windows, Linux, and macOS**.

Step 1: Create/Login to an Account

1. Go to Cisco Skills for All.
2. Create a free account or log in with your existing Cisco/Skills for All account.
3. You can also access Packet Tracer through Cisco Networking Academy when provided by your course.

Cisco documentation notes that a **NetAcad.com or SkillsForAll.com account may be required to authenticate when launching Packet Tracer**.

Step 2: Download Packet Tracer

1. Open the **Resources** section.
2. Select **Download Packet Tracer**.
3. Choose the installer appropriate for your operating system:
    - **Windows**
    - **macOS**
    - **Linux**
4. Save the installation file to your computer.

Cisco's official instructions provide installers for these desktop operating systems.

Step 3: Install on Windows

1. Open the folder where the downloaded installer is saved.
2. Double-click the Packet Tracer installation file.
3. Read and accept the **License Agreement**.
4. Click **Next**.
5. Select the installation location or keep the default location.
6. Select the Start Menu folder.
7. Choose whether to create additional shortcuts.
8. Click **Install**.
9. Wait for the installation to finish.
10. Click **Finish**.
11. Open **Cisco Packet Tracer** from the Start Menu or desktop shortcut.

These steps follow Cisco's published Windows installation procedure.

Step 4: Install on Ubuntu/Linux

1. Download the appropriate **.deb** Packet Tracer installer.
2. Open **Terminal**.
3. Go to the folder containing the downloaded file.
4. Run:

```
sudo apt-get install ./CiscoPacketTracer_xxx_Ubuntu_64bit.deb
```

Replace `xxx` with the version number of the downloaded Packet Tracer file.

5. Follow the License Agreement prompts.
6. Accept the agreement.
7. Complete the installation.
8. Launch Packet Tracer from the applications menu.

Cisco's installation instructions specifically provide the `apt-get install ./...deb` method for Ubuntu.

Step 5: Install on macOS

1. Download the macOS version of Packet Tracer.
2. Open the downloaded installer.
3. Follow the installation prompts.
4. Accept the Cisco License Agreement.
5. Complete the installation.
6. Open **Cisco Packet Tracer** from Applications.
7. Sign in using your Cisco/NetAcad/Skills for All account when prompted.

Step 6: Start the Experiment

After installation:

1. Open **Cisco Packet Tracer**.
2. Select **End Devices**.
3. Drag **4 PCs** into the workspace.
4. Select **Switches** and drag **one switch** into the workspace.
5. Select **Connections**.
6. Choose the **Copper Straight-Through** cable.
7. Connect each PC to the switch.
8. Assign IP addresses, for example:
    - PC0: `192.168.1.1`
    - PC1: `192.168.1.2`
    - PC2: `192.168.1.3`
    - PC3: `192.168.1.4`
    - Subnet mask: `255.255.255.0`
9. Use **Simulation Mode** to observe how packets travel between the computers.

RESULT:

Thus, the behaviour of network devices was analysed and studied with **CISCO PACKET TRACER SIMULATOR**.

EXP NO:4 **DATE:**

STUDY OF LAN CONFIGURATION

AIM:

To study the setup and configuration of LAN (Local Area Network) using a switch and Ethernet cables.

PROCEDURE:

1. Plan and design an appropriate network topology with respect to network requirement and equipment location.
2. **Apparatus Required:** 4 computers, a switch with 8, 16, or 24 ports, and 4 Ethernet cables.
3. Connect the computers to the switch and the Ethernet cables by plugging one end of the network cable to the computer and the other end to the switch.
4. **Assign IP address to the PCs.**
    - **a.** Hold **Windows + R** and type `ncpa.cpl`.
    - **b.** Right-click on **Ethernet → go to Properties → select Internet Protocol (TCP/IP) → go to Properties**.
    - **c.** Here change the values of **IP address and subnet mask**.
5. **Configure network switch:**
    - **a.** Ensure computer is connected to the network switch.
    - **b.** Open the web browser and type the IP address of the switch, then enter username and password to login.
    - **c.** Set up a new IP address.
6. Check the connectivity between switch and other machines by using the **ping command** in Command Prompt.
7. Select a folder and share it across computers connected in the LAN.
8. Access the shared folder in the different computer.

RESULT:

Thus, Local Area Network (LAN) setup and configuration was studied.

EXERCISE - 5

PACKET CAPTURE AND ANALYSIS USING WIRESHARK

5.1 Aim:
To create a filter to display TCP/UDP packets, inspect the packets and view the flow graph.

Procedure:
1. Select Local Area Connection in Wireshark.
2. Go to Capture -> Options and set the capture to stop after 100 packets.
3. Click Start capture.
4. Search for TCP or UDP packets using the filter/search bar.
5. Select Statistics -> Flow Graph to view the packet flow.
6. Save the captured packets.

Result:
TCP/UDP packets were filtered, inspected and the flow graph was viewed successfull

5.2 Aim:
To create a filter to display and inspect only ARP packets.

Procedure:
1. Go to Capture -> Options.
2. Select stop capture automatically after 100 packets.
3. Click Start capture.
4. Search for ARP packets using the filter/search bar.
5. Select the required ARP packet and inspect its details.
6. Save the captured packets.

Result:
ARP packets were filtered and inspected successfully.

5.3 Aim:
To create a filter to display DNS packets and provide the flow graph.

Procedure:
1. Go to Capture -> Options.
2. Select stop capture automatically after 100 packets.
3. Click Start capture.
4. Search for DNS packets using the filter/search bar.
5. Select Statistics -> Flow Graph to view the packet flow.
6. Save the captured packets.

Result:
DNS packets were filtered and the flow graph was viewed successfully.

5.4 Aim:
To create a filter to display and inspect only HTTP packets.

Procedure:
1. Select Local Area Connection in Wireshark.
2. Go to Capture -> Options.
3. Select stop capture automatically after 100 packets.
4. Click Start capture.
5. Search for HTTP packets using the filter/search bar.
6. Select the required packet and inspect its details, then save the packets.

Result:
HTTP packets were filtered and inspected successfully.

5.5Aim:
To create a filter to display and inspect only IP/ICMP packets.

Procedure:
1. Go to Capture -> Options.
2. Select stop capture automatically after 100 packets.
3. Click Start capture.
4. Search for ICMP/IP packets using the filter/search bar.
5. Select the required packet and inspect its details.
6. Save the captured packets.

Result:
IP/ICMP packets were filtered and inspected successfully.

EXERCISE – 6

ERROR DETECTION AND CORRECTION USING HAMMING CODE

AIM

To write a program to implement error detection and correction using the Hamming code concept, input a text data stream, add redundant bits, transmit the encoded data through a channel file, and verify the error correction feature.

PROCEDURE

1. Create the sender program.
2. Read the input text from input.txt.
3. Convert each character of the text into its binary representation.
4. Apply Hamming code and add the required redundant bits.
5. Store the encoded binary data in a file named channel.txt.
6. Create the receiver program and read the data from channel.txt.
7. Calculate the Hamming syndrome to detect the error position.
8. If an error is detected, correct the corresponding bit.
9. Remove the redundant bits from the corrected data.
10. Convert the remaining binary data into ASCII characters and display the original text.

The sender and receiver requirements follow the lab manual's Practical-6 specification.

SENDER CODE

```
#include <stdio.h>
#include <string.h>

void encodeCharacter(unsigned char data, FILE *channel)
{
   int h[13] = {0};
   int dataBit = 0;

   for (int i = 1; i <= 12; i++)
   {
      if (i != 1 && i != 2 && i != 4 && i != 8)
      {
         h[i] = (data >> dataBit) & 1;
         dataBit++;
      }
```

```
    }

    h[1] = h[3] ^ h[5] ^ h[7] ^ h[9] ^ h[11];
    h[2] = h[3] ^ h[6] ^ h[7] ^ h[10] ^ h[11];
    h[4] = h[5] ^ h[6] ^ h[7] ^ h[12];
    h[8] = h[9] ^ h[10] ^ h[11] ^ h[12];

    for (int i = 1; i <= 12; i++)
        fprintf(channel, "%d", h[i]);

    fprintf(channel, "\n");
}

int main()
{
    FILE *input;
    FILE *channel;
    char ch;

    input = fopen("input.txt", "r");
    channel = fopen("channel.txt", "w");

    if (input == NULL || channel == NULL)
    {
        printf("File opening error\n");
        return 1;
    }

    while ((ch = fgetc(input)) != EOF)
        encodeCharacter((unsigned char)ch, channel);

    fclose(input);
    fclose(channel);

    printf("Data encoded and stored in channel.txt\n");

    return 0;
}
```

RECEIVER CODE

```
#include <stdio.h>
#include <string.h>

int decodeCharacter(char word[], int *errorPosition)
```

```
{
    int h[13] = {0};
    int syndrome = 0;
    int data = 0;
    int bit = 0;

    for (int i = 1; i <= 12; i++)
        h[i] = word[i - 1] - '0';

    int p1 = h[1] ^ h[3] ^ h[5] ^ h[7] ^ h[9] ^ h[11];
    int p2 = h[2] ^ h[3] ^ h[6] ^ h[7] ^ h[10] ^ h[11];
    int p4 = h[4] ^ h[5] ^ h[6] ^ h[7] ^ h[12];
    int p8 = h[8] ^ h[9] ^ h[10] ^ h[11] ^ h[12];

    syndrome = p1 + (p2 * 2) + (p4 * 4) + (p8 * 8);

    *errorPosition = syndrome;

    if (syndrome >= 1 && syndrome <= 12)
        h[syndrome] ^= 1;

    for (int i = 1; i <= 12; i++)
    {
        if (i != 1 && i != 2 && i != 4 && i != 8)
        {
            data = data | (h[i] << bit);
            bit++;
        }
    }

    return data;
}

int main()
{
    FILE *channel;
    char word[20];
    int errorPosition;
    int data;
    int block = 1;
```

```
    channel = fopen("channel.txt", "r");

    if (channel == NULL)
    {
        printf("Unable to open channel.txt\n");
        return 1;
    }

    printf("Decoded Output:\n");

    while (fscanf(channel, "%12s", word) == 1)
    {
        data = decodeCharacter(word, &errorPosition);

        if (errorPosition != 0)
            printf("\nError in block %d at position %d\n",
                block, errorPosition);

        printf("%c", (char)data);
        block++;
    }

    fclose(channel);

    printf("\n");

    return 0;
}
```

OUTPUT

INPUT

Input file: input.txt

HELLO

OUTPUT

Sender Output:

Data encoded and stored in channel.txt

Receiver Output:

Decoded Output:

HELLO

Error Test Output

Decoded Output:

Error in block 2 at position 5

HELLO

RESULT

The Hamming code error detection and correction program was successfully implemented and tested.

EXPT: 7 — SLIDING WINDOW PROTOCOL

Date: 24/9/26

AIM

To implement flow control at the data link layer using Sliding Window Protocol and simulate the flow of frames from sender to receiver.

PROCEDURE

1. Start the program.
2. Enter the window size and text message.
3. Divide the message into frames, one character per frame.
4. Display the frames in the sender buffer.
5. Send frames in the sequence according to the window size.
6. Receive ACK from the receiver.
7. If the correct ACK is received, slide the window and send the next frames.
8. If NACK is received, retransmit the frames.
9. Continue until all frames are transmitted.
10. Stop the program.

PROGRAM

Sender.py

```
import time

import os

window_size = int(input("Enter window size: "))

message = input("Enter text message: ")

frames = []

for i, char in enumerate(message):

    frames.append((i, char))

next_frame = 0

while next_frame < len(frames):

    end = min(next_frame + window_size, len(frames))

    with open("sender_buffer.txt", "w") as file:
```

```
        for i in range(next_frame, end):
            file.write(f"Frame No: {frames[i][0]} Data: {frames[i][1]}\n")
    print("\nFrames sent:")
    for i in range(next_frame, end):
        print(f"Frame No: {frames[i][0]} Data: {frames[i][1]}")
    print("\nWaiting for acknowledgement...")
    time.sleep(3)
    while True:
        if os.path.exists("receiver_buffer.txt"):
            with open("receiver_buffer.txt", "r") as file:
                ack = file.read().strip()
            if ack:
                break
        time.sleep(1)
    print("Receiver response:", ack)
    if ack.startswith("ACK"):
        ack_no = int(ack.split(":")[1].strip())
        if ack_no == end:
            print(f"ACK {ack_no} received")
            print("Sliding window moved forward")
            next_frame = end
        else:
            print("Unexpected ACK")
            print("Resending current window")
```

Receiver.py

```
import time
import os
expected_frame = 1
print("Receiver started")
while True:
    if not os.path.exists("sender_buffer.txt"):
        time.sleep(1)
        continue
    with open("sender_buffer.txt", "r") as file:
        data = file.readlines()
    if not data:
        time.sleep(1)
        continue
    print("\nReceiver Frames:")
    for line in data:
        print(line.strip())
    for line in data:
        parts = line.strip().split()
        frame_no = int(parts[2])
        frame_data = parts[4]
        if frame_no == expected_frame:
            print(f"Frame {frame_no} received: {frame_data}")
            with open("receiver_buffer.txt", "w") as file:
                file.write(f"ACK:{frame_no}")
            expected_frame += 1
```

```
        else:
            print(f"Error: Expected frame {expected_frame}, but received frame {frame_no}")
            print(f"NACK {expected_frame} sent")
            with open("receiver_buffer.txt", "w") as file:
                file.write(f"NACK:{expected_frame}")
    time.sleep(1)
```

ERROR TEST / OUTPUT

The handwritten test output shows:

Error: Expected frame 2, but received frame 5

NACK 2 sent

Sender O/P:

Receiver response: NACK:2

NACK 2 received

Resending frames from the error frame.

INPUT

Enter window size: 3
Enter text message: Hello

OUTPUT — SENDER

Sender:

Frames sent:

Frame No: 1 Data: H
Frame No: 2 Data: E
Frame No: 3 Data: L

Waiting for acknowledgement...

Receiver response: ACK:3

ACK 3 received

Sliding window moved forward

OUTPUT — RECEIVER

Receiver:

Receiver started

Received frames:

Frame No: 1 Data: H

ACK 3 sent

The second page continues with the error/retransmission test.

Error Test Output

Error test O/P:

Receiver O/P:

Error: Expected frame 2, but received frame 5

NACK 2 sent

Sender O/P:

Receiver response: NACK:2

NACK 2 received

Resending frames from the error frame.

EXERCISE - 8

OS NMAP LIVE HOST DISCOVERY

Aim:

Discover live active host in a network using scans such as ARP, ICMP, TCP and UDP.

Objective:

To understand how Nmap identifies active systems before performing further network scanning.

Software:

Nmap
Try hack Nmap from attackbook computer with network access.

Theory:

Nmap is a free and open-source network scanning tool used to discover live hosts and identify network services. It was created.

* Nmap uses different live hosts using different protocols.

1) ARP Scan - uses ARP scan requests to find host on same subnet.
2) ICMP Scan - uses ICMP request and replies to identify.
3) TCP Scan - uses TCP request and packets to specified ports and check the response.
4) UDP Scan - send UDP packets and can identify an active host through an ICMP port unreachable.

Observation:

* ARP request can discover hosts within the same subnet.
* ARP cannot cross routers because it operates at the link layer.
* An ACK ping generally requires a predefined account.
* UDP ping can identify an active host when an ICMP port unreachable is received.
* The -sn option allows nmap to perform host discovery without conducting a port scan.

Result:

Thus, the live host discovery using Nmap was successfully studied and performed. The experiment helped in understanding how active hosts can be identified before performing further network scans.

EX No: 9

Date:

IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER SIMULATOR

AIM:

To implement subnetting in Cisco Packet Tracer Simulator and configure a network topology using subnetted IP addresses.

CREATE A NETWORK TOPOLOGY:

The first step in implementing classful IP subnetting is to create a network topology in Packet Tracer. Select the "New" option to create a new network. This will create a blank network topology where devices can be added.

SUBNETTING:

The subnet network address of 192.168.10.0/24 is subnetted to provide enough space for at least 5 addresses for each device/switch/router. We can use a /27 subnet mask. This gives us 8 subnets with 30 usable host addresses in each subnet.

The IP addressing for the network shown in the topology can be:

Device / Interface	IP Address
Router R – GigabitEthernet 0/0	192.168.1.1
Router R – GigabitEthernet 0/1	192.168.2.1
PC1	192.168.1.11
PC2	192.168.1.12
PC3	192.168.1.13
PC4	192.168.1.14
Fast Ethernet 0/1	192.168.3.0/27
PC1	192.168.4.11
PC2	192.168.4.12
PC3	192.168.4.13
PC4	192.168.4.14

Replace the IP address and subnet mask with your desired IP address and subnet mask.

Configure Terminal:

```
interface FastEthernet 0/1
switchport mode access
exit
interface FastEthernet 0/1
switchport mode access
exit
```

TESTING NETWORK:

Now our network topology is configured. Open a Command Prompt on each PC and try to ping the required PCs.

Last Status	Source	Destination	Type
Successful	PC4(12)	Router	ICMP
Successful	PC4(2)	PC2(10)	ICMP
Successful	PC0	Router	ICMP

Type	Color	Time	Num
ICMP	Green	0.000	12
ICMP	Blue	0.000	13
ICMP	Blue	0.000	14

RESULT:

The PCs connected to different switches can communicate with each other via routers.

The successful pings using the network properly configured show that the network is configured and operational.

EX:10

Date:

a) Internetworking with routers in Cisco Packet Tracer

Aim :

To design and configure a simple internetwork using a router in Cisco Packet Tracer simulator & verify the connectivity between the network devices.

Requirements:

1. Cisco Packet Tracer
2. 1 Router
3. 2 PCs
4. Copper straight-through cables

Procedure:

- Open Cisco Packet Tracer and place one router and two PCs in the workspace.
- Configure Router using the CLI.
- Configure FastEthernet 0/0 with IP address 192.168.10.1 and subnet mask 255.255.255.0.
- Configure FastEthernet 0/1 with IP address 192.168.20.1 and subnet mask 255.255.255.0.
- Enable both router interfaces using the no shutdown command.
- Verify that the packet is successfully transferred between PC0 and PC1.

Result:

Thus, the internetwork was successfully designed & configured using a router in Cisco Packet Tracer, connectivity between PC0 & PC1 was successfully verified.

b) Design & Configure an internetworking using wireless router, DHCP Server & Internet Cloud.

Aim :

To design and configure an internetworking using a wireless router, DHCP server & internet cloud in Cisco Packet Tracer.

Requirements:

- Cisco Packet Tracer
- Wireless Router
- DHCP server
- PCs
- Internet cloud
- Ethernet cables

Procedure:

- Open Cisco Packet Tracer and place a wireless router, DHCP server, PCs and Internet Cloud, Webserver.
- Connect the devices using appropriate Ethernet cables.
- Configure the wireless router with the required network settings.
- Configure the DHCP server to automatically assign IP address to the PCs.
- Enable DHCP service on the server.
- Configure the DNS service as given in the experiment.
- Use Ping Cisco.com from the PC to verify connectivity.

Result:

The internetwork was successfully designed & configured using a wireless router, DHCP server & internet cloud in Cisco Packet Tracer.

Date:

10: a) Internetworking with routers in Cisco Packet Tracer

b) Design & Configure an internetworking using wireless router, DHCP Server & Internet Cloud.

Ex.no: 11
Date: 9/9/26

NETWORK ADDRESS TRANSLATION (NAT) USING CISCO PACKET TRACER

Aim:

To implement Network Address Translation (NAT) in Cisco Packet Tracer and allow private devices to communicate with an outside network.

Procedure:

1) Add devices
• 2 PCs
• 1 Switch
• 1 Router
• 1 Cloud / Another Router

2) Connect devices
• PC1, PC2 → Switch
• Switch → Router G0/0
• Router G0/1 → Outside Router / Cloud

Assign IP address:

Device	IP address	Gateway
PC1	192.168.1.10	192.168.1.1
PC2	192.168.1.11	192.168.1.1
Router G0/0	192.168.1.1	—
Router G0/1	200.1.1.1	—

Configure router

Open router → CLI and enter
enable
configure terminal
interface gigabitEthernet 0/0
ip 192.168.1.1 255.255.255.0
ip nat inside
no shutdown
exit
interface gigabitEthernet 0/1
ip 200.1.1.1 255.255.255.0
ip nat outside
no shutdown
exit

Configure static NAT:

ip nat inside source static 192.168.1.10 200.1.1.10

Configure dynamic NAT:

ip nat pool My_pool 200.1.1.20 200.1.1.25
netmask 255.255.255.0
access-list 1 permit 192.168.1.0 0.0.0.255
ip nat inside source list 1 pool My_pool

Test NAT

From PC1, open command prompt and type:

ping 200.1.1.2

Check NAT translations on the router. Show IP NAT translations to verify that the translation has occurred.

Result:

NAT was successfully configured and private IP addresses were translated to public IP addresses for communication with the outside network.

Ex.no: 12

Date: 16/9/26

NMAP TO DISCOVER LIVE HOST USING ARP SCAN, ICMP SCAN, AND TCP/UDP PING SCAN IN NMAP.

AIM:

Learn and implement NMAP host discovery technique using ARP, ICMP, TCP, UDP ping to identify live hosts on a network and compare their effectiveness.

Procedure:

→ ARP scan: nmap -sn -PR 10.10.10.0/24

→ ICMP Echo/timestamp/mask: nmap -sn

→ TCP SYN/ACK (-PS, specific ports)

→ UDP: nmap -sn -PU53 10.10.10.0/24

→ Combined / disabled ARP: nmap -sn -PE

Input:

nmap -sn -PR 192.168.1.0/25 --packet-trace

-PE 10.10.X.0/24 nmap -sn -PE 10.10.X.100

nmap -sn -Pn 10.10.X.100

nmap -sn -PS22,80,443 10.10.X.0/24

nmap -sn -PA80,443 10.10.X.0/24

-PS -Pn -PU 10.10.20.0/24 nmap -sn

10.10.X.0/24 --disable-arp-ping

Target IP range (10.10.X.100 or 10.10.X.0/24)

Optional port list for -PS, -PA, -PU: 22, 80, 443

Output:

ARP

Nmap scan report for 10.10.X.1 host

0 up (10.0.0.20.1 (Gateway))

MAC Address: 02:08:75:5A:F2:XX

Nmap done: 256 IP addresses (2 hosts up)

ICMP

Nmap scan report for 10.10.X.1 host

0 up (0.00099s latency)

Nmap done: 1 IP address (1 host up)

Result:

The program was executed successfully.

Ex.no: 13

Date: 23/9/26

DEMONSTRATE NETWORK FORENSICS USING PCAPX RAY TOOL

AIM:

To analyze and visually detect network traffic from a PCAP file using the PCAPXray tool for forensic investigation.

Steps to complete network forensics experiment using PCAPXray:

1) Install Python 3 on your computer

2) Install pcapxray

pip install pcapxray

3) Get a PCAP file.

4) Open command prompt / terminal and go to the folder containing the PCAP file:

cd path-to-your-folder

5) Run PcapXray

PcapXray -f capture.pcap

6) Open the result:

• A file named pcapxray.html will be created.

• Open it in a web browser.

7) Analyze the network

• Check the connected computers.

• Check IP address.

• Check protocol and port.

• Look for unusual or suspicious connections.

Result:

The network traffic was analyzed successfully using PcapXray, and the hosts, connection protocols, and ports were identified.

WRITE A CODE USING RAW SOCKETS TO IMPLEMENT PACKET SNIFFING

EX.NO: 14

DATE: 30/10/26

Aim:

Write a code using RAW Socket to implement packet sniffing.

Algorithm:

1. Create a raw socket.
2. Bind the socket to the network interface.
3. Receive packets continuously.
4. Read the packet header.
5. Display the source and destination IP address.
6. Display the protocol used.
7. Continue until the user stops the program.

Input:

Network packets received through the network interface.

CODE:

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
s.bind(("0.0.0.0", 0))
print("Packet Sniffer Started...")

while True:
    packet, address = s.recvfrom(65535)
    print("Source IP:", address[0])
    print("Packet length:", len(packet))
    print("-" * 30)
```

OUTPUT:

```
Packet Sniffer Started...

Source IP: 192.168.1.10
Packet length: 79

------------------------------

Source IP: 192.168.1.1
Packet length: 65
```

Result:

Thus, a raw socket program was successfully implemented to capture and display network packets.

Ex. No: 15

Date: 07/10/26

ANALYSE VARIOUS TYPES OF SERVER USING WEBALIZER TOOL

AIM:

To analyze different types of web logs using the Webalizer tool.

ALGORITHM:

1. Install and open Webalizer.
2. Select the web server log file.
3. Configure Webalizer to read the log file.
4. Run Webalizer to process the logs.
5. Open the generated HTML report.
6. Analyze visitors, requests, pages, data transfer, and HTTP status codes.

INPUT:

The web server log file such as access log.

OUTPUT:

Webalizer generates a report showing:

- Number of visitors
- Number of requests
- Most visited pages
- Data transferred
- Browser and operating system
- HTTP status codes

RESULT:

The web logs were successfully analyzed using Webalizer, and the web traffic statistics were generated.